

PROGRAMMING FUNDAMENTALS

Spring 2024

8th May

Recursion

- A function that calls itself is a recursive function
- Recursion makes debugging easier

Stack

- Every time you call a function, the address to which the program should return at the completion of the function is stored in a memory location called the stack
- When a function ends, the address is retrieved from the stack and the program returns to the location from which the call was made
- The stack has a finite size → when you write useful recursive methods you must provide a way for the recursion to stop eventually

- There's no way to stop the recursive calls
- This function is like an infinite loop because there is no code to stop it from repeating

```
#include <iostream>
using namespace std;

void message()
{
    cout << "This is a recursive function.\n";
    message();
}

int main()
{
    message();

    return 0;
}
```

What is next?

- Eventually, these recursive function calls will use up all available stack memory and cause it to overflow

Can we control recursion?

- We can define number of times a function will call itself

```
void message(int c)
{
    if (c > 0)
    {
        cout << "Don't worry if it doesn't work right.\n";
        message(c - 1);
    }
}
```

Output with c = 5



D:\Books-ICT\C-Programs\recursion-1.exe

```
Enter number of times you want to repeat the message : 5
Don't worry if it doesn't work right.
Don't worry if it doesn't work right.
Don't worry if it doesn't work right.
Don't worry if it doesn't work right.
Don't worry if it doesn't work right.
-----
Process exited after 2.854 seconds with return value 0
Press any key to continue . . .
```

Using recursion for Factorial

- The factorial of a nonnegative number can be defined by the following rule:

$$n ! = 1 \times 2 \times 3 \times \dots \times n$$

We also know that

$$0! = 1$$


```
int factorial(int n)
{
    if (n == 0)
        return 1; // Base case
    else
        return n * factorial(n - 1); // Recursive case
}
```

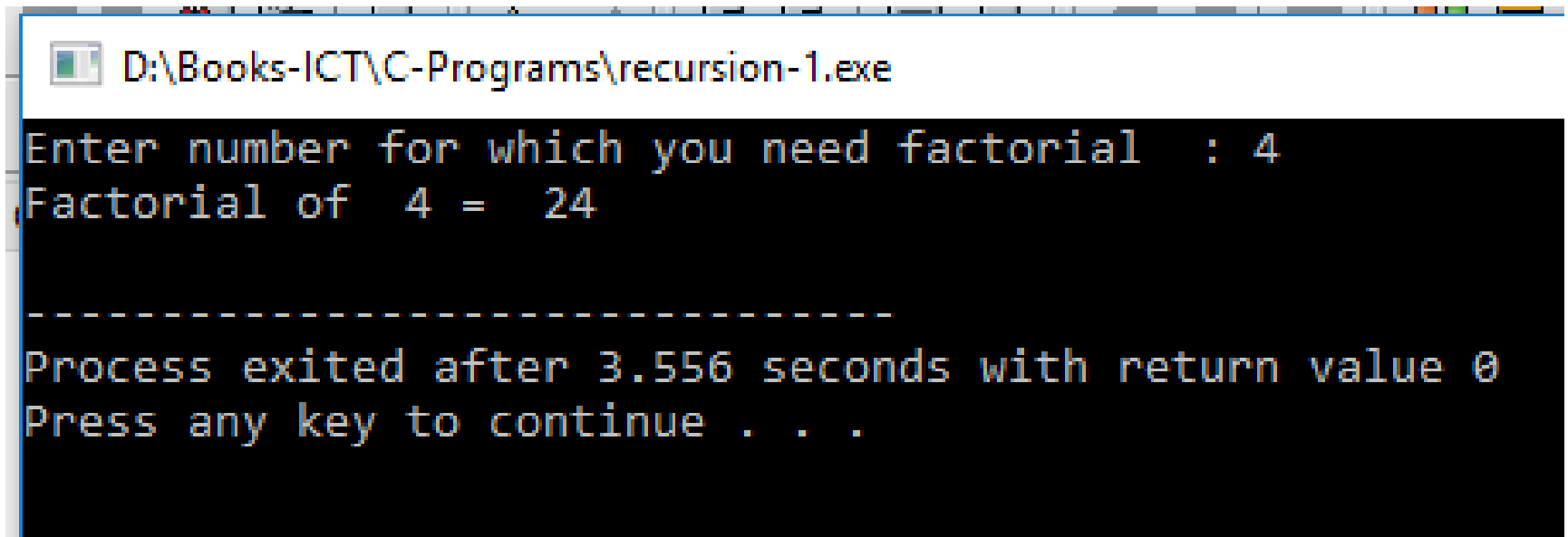
```
int main()
{
    int num;
    cout<<"Enter number for which you need factorial : " ;
    cin>>num;

    cout<<"Factorial of " <<num<<" = " <<factorial(num)<<endl;

    return 0;
```



Run Window



A screenshot of a Windows Run Window titled "D:\Books-ICT\C-Programs\recursion-1.exe". The window has a black background with yellow text. The text inside the window reads: "Enter number for which you need factorial : 4", "Factorial of 4 = 24", a dashed line separator, "Process exited after 3.556 seconds with return value 0", and "Press any key to continue . . .".

```
D:\Books-ICT\C-Programs\recursion-1.exe
Enter number for which you need factorial : 4
Factorial of 4 = 24
-----
Process exited after 3.556 seconds with return value 0
Press any key to continue . . .
```

main()

```
int main()
{
    int num;
    char my_letter;
    char my_chr;
    string my_str;
    int str_len;
    cout<<"Enter a string : " ;
    cin>>my_str;
    cout<<"Enter letter to find: ";
    cin>>my_letter;
    cout << "your entered letter appears " << numChars(my_letter, my_str, 0) << " times.\n";
    return 0;
}
```

Recursive function

```
int numChars(char c, string str, int position)
{
    if (position >= str.length())
    {
        // Base case: The end of the string is reached.
        return 0;
    }
    else if (str[position] == c)
    {
        return 1 + numChars(c, str, position+1);
        // Recursive case: A matching character was found.
        // Return 1 plus the number of times the search
        // character appears in the rest of the string.
    }
    else
    {
        return numChars(c, str, position+1);
        // Recursive case: A character that does not match the
        // search character was found. Return the number of times
        // the search character appears in the rest of the string.
    }
}
```

Run Window

 D:\Books-ICT\C-Programs\recursion-1.exe

Enter a string : shaziarizwan

Enter letter to find: a

your entered letter appears 3 times.

Process exited after 9.734 seconds with return value 0

Press any key to continue . . .

Increment Operator

- Pre-increment: `++ variable`
- Post-increment: `variable++`

Decrement Operator

- Pre-decrement: `-- variable`
 - Post-decrement: `variable--`
-
- These are called Unary operators.

Example-1

x = 5;

y = ++x;

- First statement assigns value **5** to **x**
- In the second statement, first the value of **x** is incremented to **6**, and then this value, **6**, is assigned to **y**
- So both **x** and **y** have the value **6**
- The equivalent results would occur with this set of statements:

x = 5;

x = x + 1;

y = x;

Example-2

x = 5;

y = x++;

- The second statement, first the value of **x**, which is **5**, is used to evaluate the expression, and then the value of **x** is incremented to **6**
- So **5** is stored in **y**
- After the second statement executes, the value of **x** is **6**, and the value of **y** is **5**
- The equivalent set of statements:

x = 5;

y = x;

x = x + 1;

Example-3

Lec19-Examples.cpp

```
1  #include <iostream>
2  using namespace std;
3  int main ()
4  {
5      int a,b;
6      a = 5;
7      b = 2 + (++a);
8      cout<<"a = "<<a<<endl;
9      cout<<"b = "<<b<<endl;
10     return 0;
11 }
```

Output

```
a = 6
```

```
b = 8
```

```
-----
```

```
Process exited after 0.1297 seconds with  
Press any key to continue . . .
```

Example -4

```
#include <iostream>
using namespace std;
int main ()
{
    int a,b;
    a = 5;
    b = 2 + (a++);
    cout<<"a = "<<a<<endl;
    cout<<"b = "<<b<<endl;
    return 0;
}
```

Output

```
a = 6  
b = 7  
-----  
Process exited after 0.1481 seconds  
Press any key to continue . . .
```

Example-5

```
int age = 49;  
if (age++ > 49)  
    cout << "Congratulations - You have made it to the half-century"  
        << " mark !" << endl;
```

Output

- the cout statement will not execute
- The comparison between age and 49 is made *first*
- Then the value of age is incremented by one
- Since 49 is not greater than 49, the condition is false

Example-6

```
int age = 49;  
if (++age > 49)  
    cout << "Congratulations - You have made it to the half-century"  
        << " mark !" << endl;
```


Output

 D:\Books-ICI\C-Programs\Lec19-Examples.exe

```
Congratulations - You have made it to the half-century mark !
```

```
-----
```

```
Process exited after 0.1137 seconds with return value 0
```

```
Press any key to continue . . .
```

Example-7

```
int a=5,b=2;  
a = -50;  
b = +50;  
cout<<"a = "<<a<<endl;  
cout<<"b = "<<b<<endl;
```

Output



D:\Books-ICT\C-Programs\Lec19-Examples.exe

```
a = -50
```

```
b = 50
```

```
-----  
Process exited after 0.1118 seconds with r  
Press any key to continue . . .
```

Exmple-8

```
int n = 4, k = 2;
cout << "n= " << ++n << endl;
cout << "n= " << n << endl;
cout << "n= " << n++ << endl;
cout << "n= " << n << endl;
cout << "n= " << --n << endl;
cout << "n= " << n << endl;
cout << "n= " << --n << endl;
cout << "n= " << n << endl;
cout << "n= " << n-- << endl;
cout << "n= " << n << endl;
cout << "n + k = " << n + k << endl;
cout << "n= " << n << endl;
cout << "k = " << k << endl;
cout << "n= " << n << " k= " << k << endl;
cout << "n " << n << endl;
cout << " n" << endl;
cout << "\n" << endl;
cout << " n * n = " << endl;
cout << n * n << endl;
cout << 'n' << endl;
return 0;
```

Output

```
D:\Books-ICT\C-Programs\Lec19-  
n= 5  
n= 5  
n= 5  
n= 6  
n= -6  
n= 6  
n= 5  
n= 5  
n= 5  
n= 4  
n + k = 6  
n= 4  
k = 2  
n= 4 k= 2  
n 4  
n  
  
n * n =  
16  
n  
  
-----  
Process exited after 0.112  
Press any key to continue
```

Simple Assignment Statements

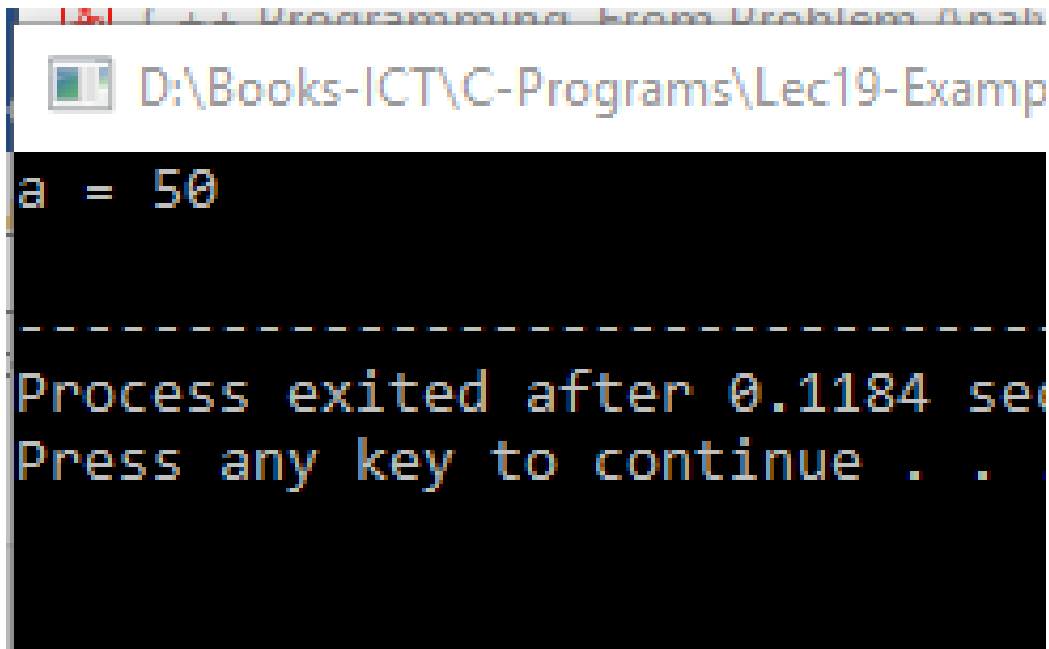
```
a = a * b;
```

```
{  
  a = a + a;  
  b = b + b;  
  c = c + c;  
}
```

Example-9

```
int a,b;  
a = 5;  
b = 10;  
a = a * b;  
cout<<"a = "<<a<<endl;
```

Output..



The screenshot shows a Windows command prompt window with a title bar that includes the file path "D:\Books-ICT\C-Programs\Lec19-Examp". The command prompt displays the output of a program, which includes the variable assignment "a = 50", a dashed line separator, and a message indicating the process has exited after 0.1184 seconds, followed by a prompt to press any key to continue.

```
a = 50  
-----  
Process exited after 0.1184 sec  
Press any key to continue . . .
```


Compound Assignment Statements

```
int a,b;  
a = 5;  
b = 10;  
a *= b;  
    cout<<"a = "<<a<<endl;
```

output

 D:\Books-ICT\C-Programs\Lec19-Examp

```
a = 50
```

```
-----  
Process exited after 0.1205 se  
Press any key to continue . .
```

Compound statement

You can write:

```
variable = variable * (expression);
```

as:

```
variable *= expression;
```

Compound statement

You can write:

```
variable = variable + (expression);
```

as:

```
variable += expression;
```

Converting simple → compound

1. `counter = counter + 2;`
2. `sum = sum + num;`
3. `amount = amount * (interest + 1);`
4. `x = x / (y + 5);`

Solution

1. `counter += 2;`
2. `sum += num;`
3. `amount *= interest + 1;`
4. `x /= y + 5;`

Data Type : Revisited

- So far we know that a data type defines a set of values together with a set of operations
- The set of values is called the domain of the data type
- The name of the data type defines the set of values belonging to this data type
- E.g. `int` data type includes integers between -2147483648 and 2147483647
- Allowed operations allowed are the arithmetic operators you learned earlier

Pointer data type

- The values belonging to pointer **data types** are the memory addresses of your computer
- There is no name associated with the pointer data type in C++ like other languages
- The domain—the set of values of a pointer data type → the addresses (locations) in memory
- **Pointer variable:** A variable whose content is a memory → is said to point to a memory address

Declaring a pointer variable

- If no name is associated with a pointer data type, how do you declare pointer variables?

Declaring a pointer variable

- When you declare a pointer variable, you also specify the data type of the value to be stored in the memory location pointed to by the pointer variable
- For example, if a pointer variable contains the address of a memory location containing an `int` value, it is said to be an `int` pointer or a pointer (variable) of type `int`.

Declaring a pointer variable

- In C++, you declare a pointer variable by using the asterisk symbol (*) between the data type and the variable name
- The general syntax to declare a pointer variable is:

dataType *identifier;

Examples

```
int *p;
```

```
char *ch;
```

- In these statements, both **p** and **ch** are pointer variables. The content of **p** points to a memory location of type **int**
- The content of **ch** points to a memory location of type **char**
- **p** is a pointer variable of type **int**
- **ch** is a pointer variable of type **char**

Explained further...

`int *p;`

- is equivalent to the statement:

`int* p;`

- which is equivalent to the statement:

`int * p;`

- Thus, the character `*` can appear anywhere between the data type name and the variable name

`int* p, q;`

- only `p` is the pointer variable, not `q`. Here, `q` is an `int` variable
- Each variable must have its own `*` character placed to the left of it to make it a pointer variable.

- To avoid confusion, it is preferred to attach the `*` to the variable name

```
int *p, q;
```

```
int *p, *q;
```

- declares both `p` and `q` to be pointer variables of type `int`

Pointers

- Pointers are a special type of variables in which a memory address is stored
- They contain a memory address, not the value of the variable

Swapping 2 values

- To swap the values of two variables x and y
- Use a third variable (temp):

```
temp = x ;
```

```
x = y ;
```

```
y = temp ;
```


Let's use pointers

- Write a swap function using pass by reference

```
swap (int *yptr, int *xptr)
{
... ..
}
```

- Prototype of function swap

```
void swap(int *, int *) ;
```

Step-1

```
#include <iostream>
using namespace std;
/* Prototye of function swap used to swap two values */
void swap(int *, int *) ;
main()
{
int x [] = {1,3,5,7,9,2,4,6,8,10};
```

Step-2

```
int i, j, tmp, swaps;
```

Step-3

```
for(i = 0; i < 10; i ++)  
{  
  
    for(j = 0; j < 10; j ++)  
    {  
        if ( x[j] > x[j+1]) //  
        {  
  
            swap(&x[j],&x[j+1]);  
        }  
    }  
}
```

.....

Step-4

```
//display the array's elements after each comparison  
for (j=0; j<10; j++)  
    cout << x[j] << '\t';  
    cout << endl;  
if (swaps == 0)  
    break;
```

Step-5

```
}  
return 0;  
}
```

Function definition

```
void swap(int *x, int *y)
{
    int tmp;
    if(*x > *y)
    {
        tmp = *x;
        *x = *y;
        *y = tmp;
    }
}
```

Call Statement

- When an element in the array is greater than the next element and swapping is needed
- X is the array having integer values

```
int x [] = {1,3,5,7,9,2,4,6,8,10};
```

- Passing addresses of the values so actual values in the array are swapped

```
swap(&x[j],&x[j+1]);
```


Output

 D:\Books-ICT\C-Programs\Bubble-Sor-PassbyRef.exe

```
1      3      5      7      2      4      6      8      9      10
1      3      5      2      4      6      7      8      9      10
1      3      2      4      5      6      7      8      9      10
1      2      3      4      5      6      7      8      9      10
1      2      3      4      5      6      7      8      9      10
```

```
-----
Process exited after 0.1732 seconds with return value 0
Press any key to continue . . .
```

```
int x [] = {1,3,5,7,9,2,4,6,8,10};
```